

VM & Docker

Documentation technique — Leon Camuzat

JOB01	Création de la VM Debian 13 sous VMware
JOB02	Lancer Docker + commandes de base
JOB03	Créer une image personnalisée (Hello World)
JOB04	Image Docker avec SSH
JOB05	Connexion SSH dans le conteneur
JOB05+	Alias Docker dans .bashrc
JOB06	Volumes Docker — stockage persistant
JOB07	Docker Compose — nginx + FTP
JOB08	Image nginx personnalisée
JOB09	Registry Docker local + Registry UI
JOB10	Scripts install/uninstall Docker
JOB11	Portainer + alternatives

JOB 01

Création de la VM Debian 13

Etape 01 Création de la VM sous VMware Workstation

Création d'une VM sous VMware Workstation avec Debian 13. Lors de la sélection des logiciels, seuls **Serveur SSH** et **Utilitaires usuels du système** sont cochés — pas d'interface graphique pour une installation serveur légère.

Etape 02 Installation des dépendances

Connexion à la VM et installation des prérequis pour Docker :

```
apt install -y ca-certificates curl gnupg
```

Etape 03 Créer le dossier pour les clés GPG

```
install -m 0755 -d /etc/apt/keyrings
```

Etape 04 Télécharger la clé GPG Docker

```
curl -fsSL https://download.docker.com/linux/debian/gpg |  
gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

Etape 05 Créer le dépôt Docker

```
echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.gpg]  
https://download.docker.com/linux/debian trixie stable"  
> /etc/apt/sources.list.d/docker.list
```

Etape 06 Installer Docker

```
apt install -y docker-ce docker-ce-cli containerd.io
```

JOB 02

Lancer Docker + commandes de base

Etape 01 Lancer Docker et tester l'installation

```
docker run hello-world
```

```
# Docker télécharge l'image hello-world et l'exécute
```

```
# 'Hello from Docker!' confirme que l'installation fonctionne
```

Etape 02 Connaître les commandes de base

Concepts fondamentaux :

Image : template immuable contenant le système et l'application. Elle ne change jamais une fois construite.

Conteneur : processus isolé basé sur cette image, exécuté grâce au moteur Docker. On peut en créer plusieurs depuis la même image.

Commandes essentielles :

docker images	Voir toutes les images stockées localement
docker ps	Voir les conteneurs en cours d'exécution
docker ps -a	Voir tous les conteneurs (même arrêtés)
docker rm ID	Supprimer un conteneur (arrêté)
docker rm -f ID	Forcer la suppression (même si actif)
docker rmi NOM_IMAGE	Supprimer une image
docker run IMAGE	Créer et lancer un conteneur depuis une image
docker --version	Vérifier la version de Docker installée

JOB 03

Créer une image personnalisée — Hello World

Création d'une image Docker basée sur Debian qui affiche un message personnalisé.

```
mkdir job03
cd job03
nano Dockerfile
```

Contenu du Dockerfile :

```
FROM debian:stable-slim
# Utilise l'image Debian minimale comme base

CMD ["echo", "Hello World depuis mon image Debian personnalisée !"]
# Lance la commande echo au démarrage du conteneur
```

Construction et lancement :

```
docker build -t my-hello .
# -t my-hello : donne le nom 'my-hello' à l'image
# . : utilise le Dockerfile du répertoire courant

docker run my-hello
# Affiche : Hello World depuis mon image Debian personnalisée !
```

Instruction FROM : définit l'image de base. **CMD** : commande exécutée au démarrage du conteneur. Le point . dans docker build = répertoire courant.

JOB 04

Image Docker avec SSH

Création d'une image Docker contenant un serveur SSH permettant de se connecter au conteneur depuis l'extérieur.

Dockerfile :

```
FROM debian:stable-slim

# Installer SSH
RUN apt update && apt install -y openssh-server && mkdir -p /var/run/sshd

# Définir mot de passe root
RUN echo 'root:root123' | chpasswd

# Autoriser login root
RUN sed -i 's/#PermitRootLogin.*/PermitRootLogin yes/' /etc/ssh/sshd_config && \
sed -i 's/#PasswordAuthentication.*/PasswordAuthentication yes/' /etc/ssh/sshd_config && \
sed -i 's/UsePAM yes/UsePAM no/' /etc/ssh/sshd_config

# Exposer port 22
EXPOSE 22

# Lancer SSH en foreground
CMD ["/usr/sbin/sshd", "-D"]
```

Construction de l'image :

```
docker build -t my-ssh .
docker images

# On voit : hello-world, my-hello, my-ssh
```

EXPOSE : documente le port utilisé (ne l'ouvre pas sur l'hôte). **RUN** : exécute une commande pendant la construction de l'image. **-D** dans sshd : mode daemon foreground (nécessaire dans Docker).

Etape 01 Lancer le conteneur SSH

```
docker run -d -p 2222:22 --name ssh-container my-ssh

# -d : mode détaché (en arrière-plan)
# -p 2222:22 : redirige le port 2222 de l'hôte vers le port 22 du conteneur
# --name ssh-container : nomme le conteneur
```

Etape 02 Se connecter en SSH

```
ssh root@localhost -p 2222

# Mot de passe : root123
# Accepter l'empreinte avec 'yes' à la première connexion

ps aux | grep sshd

# Vérifier que le daemon SSH tourne dans le conteneur
```

Etape 03 Créer des alias Docker dans .bashrc

Pour gagner du temps, on crée des alias dans `~/.bashrc` :

```
nano ~/.bashrc

# Ajouter à la fin du fichier :

alias dps='docker ps'
alias dpsa='docker ps -a'
alias di='docker images'
alias drm='docker rm -f'
alias dri='docker rmi'
alias dstart='docker start'
alias dstop='docker stop'
alias dlogs='docker logs -f'
alias dexec='docker exec -it'
alias drun='docker run -it --rm'
```

Tableau des alias :

Alias	Commande réelle	Sert à quoi ?	Quand l'utiliser ?
dps	docker ps	Voir les conteneurs actifs	Vérifier ce qui tourne
dpsa	docker ps -a	Tous les conteneurs	Debug / nettoyage
di	docker images	Voir les images locales	Vérifier les images
drm	docker rm -f	Supprimer un conteneur	Nettoyer
dri	docker rmi	Supprimer une image	Nettoyer les images
dstart	docker start	Redémarrer un conteneur	Relancer un service
dstop	docker stop	Arrêter un conteneur	Stopper un service
dlogs	docker logs -f	Logs en temps réel	Debug d'un conteneur
dexec	docker exec -it	Entrer dans un conteneur	Administration interne

<code>drun</code>	<code>docker run -it --rm</code>	Conteneur temp interactif	Test rapide
-------------------	----------------------------------	---------------------------	-------------

JOB 06

Volumes Docker — Stockage persistant

Volume Docker : stockage de données indépendant des conteneurs. Plusieurs conteneurs peuvent monter le même volume pour partager des données. Les volumes sont **persistants** : ils survivent à la suppression des conteneurs.

```
# Créer un volume nommé 'partage'
docker volume create partage

# Lister les volumes
docker volume ls

# DRIVER VOLUME NAME
# local partage

# Conteneur c1 : écrire dans le volume
docker run -it --name c1 -v partage:/data debian bash
echo 'Test volume OK' > /data/test.txt
exit

# Conteneur c2 : lire depuis le même volume
docker run -it --name c2 -v partage:/data debian bash
cat /data/test.txt

# Affiche : Test volume OK -- les données ont été partagées !
exit

# Supprimer les conteneurs (le volume reste !)
docker rm -f c1 c2
docker volume ls

# Le volume 'partage' est toujours là
```


Docker Compose permet de définir et lancer plusieurs conteneurs depuis un fichier YAML. Ici on déploie un serveur web nginx et un serveur FTP partageant le même volume.

Fichier `docker-compose.yml` :

```
services:
  nginx:
    image: nginx:alpine
    container_name: nginx
    ports:
      - "8080:80"
    volumes:
      - webdata:/usr/share/nginx/html
    depends_on:
      - ftp
  ftp:
    image: fauria/vsftpd
    container_name: ftp
    ports:
      - "2121:21"
      - "30000-30009:30000-30009"
    environment:
      - FTP_USER=leon
      - FTP_PASS=leon123
      - PASV_ADDRESS=192.168.163.129
      - PASV_MIN_PORT=30000
      - PASV_MAX_PORT=30009
    volumes:
      - webdata:/usr/share/nginx/html
  volumes:
    webdata:
```

Lancer et vérifier :

```
docker compose up # lancement en avant-plan (voir les logs)
docker compose up -d # lancement en arrière-plan (détaché)
docker ps # vérifier les 2 conteneurs (nginx + ftp)

# nginx accessible sur http://192.168.163.129:8080
# FTP accessible sur le port 2121 (user: leon, pass: leon123)
```

JOB 08

Image nginx personnalisée

Création d'une image nginx personnalisée avec un fichier HTML custom (Leon Camuzat). L'image est construite depuis un Dockerfile.

Dockerfile :

```
FROM debian:stable-slim

RUN apt update && apt install -y nginx && apt clean

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

# daemon off : nginx tourne en foreground (requis pour Docker)
```

```
docker build -t my-nginx .

docker run -d -p 8090:80 --name nginx-container my-nginx

# Tester depuis la VM

curl http://localhost:8090

# Affiche le HTML de la page nginx personnalisée

# Accessible depuis le navigateur sur :

# http://192.168.163.129:8090
```

JOB 09

Registry Docker local + Registry UI

Déploiement d'un registre Docker privé local (comme Docker Hub mais en local) avec une interface web pour le visualiser.

docker-compose.yml — Registry :

```
services:
  registry:
    image: registry:2
    container_name: registry
    ports:
      - '5000:5000'
    volumes:
      - registry-data:/var/lib/registry
    restart: always

  registry-ui:
    image: joxit/docker-registry-ui:latest
    container_name: registry-ui
    ports:
      - '8081:80'
    environment:
      - REGISTRY_TITLE:Mon Registry Local
      - REGISTRY_URL=http://registry:5000
      - SINGLE_REGISTRY=true
    depends_on:
      - registry

  volumes:
    registry-data:
```

```
docker compose up -d

# Tagger et pousser une image dans le registry local
docker tag my-nginx localhost:5000/my-nginx:v1
docker push localhost:5000/my-nginx:v1

# Vérifier le contenu du registry
curl http://localhost:5000/v2/_catalog

# {"repositories":["my-nginx"]}

# Interface web accessible sur : http://192.168.163.129:8081
```

JOB 10

Scripts install/uninstall Docker

Création de deux scripts bash pour automatiser l'installation et la désinstallation complète de Docker.

Script `uninstall_docker.sh` :

```
#!/bin/bash

echo "Arret des conteneurs..."

docker stop $(docker ps -aq) 2>/dev/null
docker rm -f $(docker ps -aq) 2>/dev/null

echo "Suppression des images..."

docker rmi -f $(docker images -aq) 2>/dev/null

echo "Suppression des volumes..."

docker volume rm $(docker volume ls -q) 2>/dev/null
docker network prune -f 2>/dev/null

echo "Desinstallation des paquets..."

apt-get purge -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
apt-get autoremove -y

rm -rf /var/lib/docker /var/lib/containerd /etc/docker
rm -f /etc/apt/sources.list.d/docker.list /etc/apt/keyrings/docker.gpg

apt-get update

echo "Docker completement supprime !"
```

Script `install_docker.sh` :

```
#!/bin/bash

echo "Mise a jour du systeme..."

apt-get update && apt-get upgrade -y

apt-get install -y ca-certificates curl gnupg lsb-release

echo "Ajout de la cle GPG..."

install -m 0755 /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/debian/gpg | gpg --dearmor -o
/etc/apt/keyrings/docker.gpg

chmod a+r /etc/apt/keyrings/docker.gpg

echo "Ajout du depot Docker..."

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/debian $(lsb_release -cs) stable" | tee
/etc/apt/sources.list.d/docker.list

apt-get update

echo "Installation de Docker..."

apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin

systemctl enable docker && systemctl start docker

echo "Docker Installe !"

docker --version
```

```
# Rendre les scripts exécutable  
chmod +x install_docker.sh uninstall_docker.sh  
  
# Exécuter l'installation  
./install_docker.sh
```

JOB 11

Portainer — Interface graphique Docker

Portainer est une interface web permettant de gérer Docker visuellement : conteneurs, images, volumes, réseaux et stacks Compose.

Installation de Portainer CE :

```
# Créer le volume de données Portainer
docker volume create portainer_data

# Lancer le conteneur Portainer
docker run -d \
  -p 8000:8000 \
  -p 9443:9443 \
  --name portainer \
  --restart=always \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v portainer_data:/data \
  portainer/portainer-ce:latest

# Interface accessible sur : https://192.168.163.129:9443
# Identifiants : admin / ilfaitbeauamarseille
```

docker.sock : fichier socket Unix permettant à Portainer de communiquer directement avec le daemon Docker de l'hôte. **--restart=always** : redémarre automatiquement au boot.

Alternatives à Portainer :

Outil	Difficulté	Interface	Points forts
Portainer	Moyen	Web	Complet, très populaire
Yacht	Facile	Web	Simple et moderne, idéal débutant
Lazydocker	Facile	Terminal	Léger, parfait sans GUI
Dockge	Facile	Web	Idéal pour gérer les Compose
Rancher	Difficile	Web	Entreprise / Kubernetes
Cockpit	Moyen	Web	Gestion système complète + Docker
Docker Desktop	Facile	App	Windows/Mac uniquement

Lancer Lazydocker (alternative terminal) :

```
docker run -it --rm -v /var/run/docker.sock:/var/run/docker.sock lazyteam/lazydocker
```